

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES

De Béni Mellal

TP 1

Application Monolithique avec Spring Boot

E-Commerce

Réalisé par :

Bah Thierno Mamoudou

Encadrant :

Pr. BE ELBAGHAZAOUI

Année universitaire : 2025 — 2026

1. Introduction et Objectifs

Ce travail pratique a pour but de concevoir et de mettre en œuvre une application web monolithique à l'aide du framework Spring Boot. L'application réalisée permet la gestion d'un catalogue de produits e-commerce au travers d'une API REST complète respectant les standards HTTP.

Un monolithe est une architecture dans laquelle l'ensemble des fonctionnalités de l'application (interface, logique métier, accès aux données) est regroupé au sein d'un seul et unique déploiement. Cette approche est particulièrement adaptée aux projets de petite et moyenne taille, car elle simplifie le développement, le débogage et la mise en production.

Objectifs du TP

- Créer une application Spring Boot monolithique structurée en couches.
- Développer une API REST CRUD complète (GET, POST, PUT, DELETE).
- Comprendre l'architecture en couches : Entity, Repository, Service, Controller.
- Utiliser `@RepositoryRestResource` pour générer automatiquement des endpoints REST.
- Réaliser l'exercice d'extension : ajout de l'entité Category avec relation `@ManyToOne`.
- Tester l'ensemble des fonctionnalités via Postman.

Outils et Technologies utilisés

Outil / Technologie	Version	Rôle
Java / JDK	21	Langage principal
Spring Boot	3.x	Framework applicatif
Spring Data JPA	3.x	Accès aux données
Spring Data REST	3.x	API REST automatique
PostgreSQL	—	Base de données
Lombok	—	Réduction du boilerplate
IntelliJ IDEA	—	Environnement de développement
Postman	—	Tests des endpoints REST
Maven	4.0	Gestion des dépendances

2. Architecture du Projet

L'application adopte une architecture en couches (Layered Architecture), principe fondamental du développement Spring Boot. Chaque couche possède une responsabilité unique et bien délimitée, favorisant la maintenabilité et la testabilité du code.

Les 4 couches de l'application

Couche	Classe principale	Rôle
Entité (Model)	Product.java	Représente la table SQL "products", portant les annotations JPA et les validations Jakarta.
Repository	ProductRepository.java	Interface étendant JpaRepository, fournissant les opérations CRUD. Utilise @RepositoryRestResource pour exposer une API automatique.
Service	ProductService.java	Contient la logique métier. Gère les transactions avec @Transactional et délègue la persistance au Repository.
Contrôleur	ProductController.java	Expose les endpoints REST sur /api/products. Reçoit les requêtes HTTP et retourne des ResponseEntity typées.

Cette organisation respecte le principe de séparation des préoccupations (Separation of Concerns) : le contrôleur ne connaît pas la base de données, le service ne connaît pas HTTP, et le repository n'applique aucune logique métier.

3. Tests des Endpoints REST avec Postman

L'ensemble des endpoints REST a été testé avec Postman. L'application est démarrée localement sur le port 8080. Voici les résultats obtenus pour chaque opération CRUD.

3.1 GET — Liste de tous les produits

La requête GET sur /api/products retourne la liste complète des produits enregistrés en base. Le code de réponse est 200 OK.

The screenshot shows a REST client interface. At the top, the method is **GET** and the URL is `{{base_url}}/api/products`. Below the URL bar, there are tabs for **Docs**, **Params**, **Authorization**, **Headers (6)**, **Body**, **Scripts**, and **Settings**. The **Params** tab is selected, showing a table for **Query Params** with columns **Key**, **Value**, and **Description**. The table is empty.

Below the tabs, the **Body** tab is selected, showing the response status **200 OK** with a response time of **441 ms** and a size of **507 B**. The response is in **JSON** format. The JSON body is a list of two products:

```
3     "id": 2,
4     "name": "Telephone",
5     "description": "Telephone de grande valeur",
6     "price": 3200.00,
7     "stock": 10,
8     "category": null
9   },
10  {
11    "id": 3,
12    "name": "Laptop",
```

Figure 1 — GET /api/products : réponse 200 OK avec la liste des produits

3.2 POST — Création d'un produit

La requête POST sur /api/products avec un corps JSON permet de créer un nouveau produit. Dans cet exemple, Refrigerateur est créé avec un prix de 7500 et un stock de 5. La réponse attendue est 201 Created.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `{{base_url}}/api/products`
- Body Type:** raw (selected)
- Request Body (JSON):**

```
1 {
2   "name": "Refrigerateur",
3   "description": "Taille moyenne",
4   "price": 7500.00,
5   "stock": 5,
6   "category": {
7     "id" : 2
8   }
}
```
- Response:** 201 Created
- Response Body (JSON):**

```
1 {
2   "id": 6,
3   "name": "Refrigerateur",
4   "description": "Taille moyenne",
5   "price": 7500.00,
6   "stock": 5,
7   "category": {
8     "id": 2,
```

Figure 2 — POST `/api/products` : création d'un produit (Refrigerateur)

3.3 PUT — Mise à jour d'un produit

La requête PUT sur `/api/products/{id}` permet de mettre à jour intégralement un produit existant. Ici, le produit d'ID 3 est mis à jour avec le nom "Laptop pro", un prix de 4500 et un stock de 7. La réponse est 200 OK avec les données mises à jour.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** `{{base_url}}/api/products/3`
- Body:**

```
{
  "name": "Laptop Pro",
  "description": "Updated version",
  "price": 4500,
  "stock": 7,
  "category": {
    "id": 1
```
- Response:** 200 OK (39 ms)
- Response Body:**

```
{
  "id": 3,
  "name": "Laptop Pro",
  "description": "Updated version",
  "price": 4500,
  "stock": 7,
  "category": {
    "id": 1,
    "name": "Electronics"
```

Figure 3 — PUT `/api/products/7` : mise à jour (200 OK)

3.4 DELETE — Suppression d'un produit

La requête DELETE sur `/api/products/{id}` supprime le produit correspondant. La réponse est 204 No Content, confirmant que la suppression a bien eu lieu sans corps de réponse.

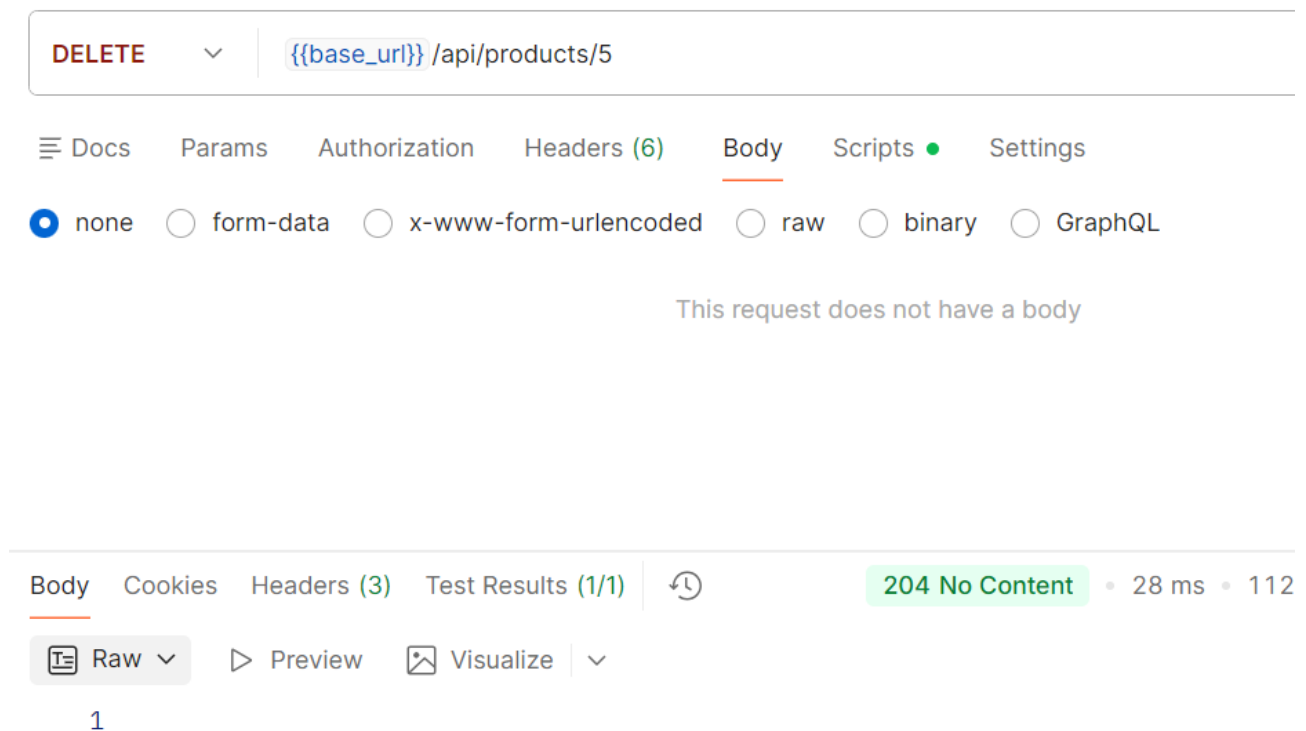


Figure 4 — DELETE /api/products/2 : suppression (204 No Content)

4. Tests Postman — Category

Création d'une catégorie

Un POST sur /categories avec le corps `{"name": "Habits"}` crée la catégorie. La réponse 201 Created inclut les liens HATEOAS générés automatiquement par Spring Data REST.

POST `{{base_url}}/categories`

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL

```
1 {
2   "name": "Habits"
3 }
```

Body Cookies Headers (6) Test Results (1/1) 201 Created • 179

`{}` JSON Preview Visualization

```
1 {
2   "_links": {
3     "self": {
4       "href": "http://localhost:8080/categories/3"
5     },
6     "category": {
7       "href": "http://localhost:8080/categories/3"
8     }
9   },
10  "name": "Habits"
11 }
```

Figure 12 — POST /categories : création de la catégorie SMARTPHONES (201 Created)

Création d'un produit lié à une catégorie

En ajoutant "category": {"id": 1} dans le corps du POST /api/products, le produit est associé à la catégorie créée précédemment. La réponse 201 Created confirme la création avec la relation ManyToOne persistée.

The screenshot displays a REST client interface with the following components:

- Method and URL:** A dropdown menu shows **POST** and a text field contains `{{base_url}}/api/products`.
- Navigation Tabs:** Docs, Params, Authorization, Headers (8), **Body** (selected), Scripts, and Settings.
- Body Type Selection:** Radio buttons for none, form-data, x-www-form-urlencoded, **raw** (selected), binary, and GraphQL.
- Request Body:** A code editor showing a JSON object:

```
1  {
2    "name": "Refrigerateur",
3    "description": "Taille moyenne",
4    "price": 7500.00,
5    "stock": 5,
6    "category": {
7      "id" : 2
8    }
9  }
```
- Response Section:**
 - Tabs: Body, Cookies, Headers (5), Test Results (1/1), and a refresh icon.
 - Status: A green box displays **201 Created**.
 - Content Type: A dropdown shows **{ } JSON**.
 - Actions: Preview and Visualization buttons.
 - Response Body:** A code editor showing the returned JSON:

```
1  {
2    "id": 6,
3    "name": "Refrigerateur",
4    "description": "Taille moyenne",
5    "price": 7500.00,
6    "stock": 5,
7    "category": {
8      "id": 2,
```

Figure 13 — POST /api/products : création d'un produit associé à la catégorie (201 Created)

5. Conclusion et Bilan

Ce TP a permis de mettre en pratique les concepts fondamentaux du développement d'applications web avec Spring Boot. L'architecture en couches adoptée garantit une séparation claire des responsabilités et facilite la maintenance du code à long terme.

Compétences acquises

- Initialisation d'un projet Spring Boot via Spring Initializr avec les dépendances adaptées.
- Conception d'une entité JPA avec validations Jakarta et annotations Lombok.
- Utilisation de `@RepositoryRestResource` pour exposer une API REST sans écrire de contrôleur.
- Implémentation d'un service transactionnel avec gestion des erreurs (`orElseThrow`).
- Rédaction d'un contrôleur REST complet avec les bonnes pratiques de codes HTTP.
- Modélisation d'une relation bidirectionnelle `@ManyToOne` / `@OneToMany` entre deux entités.
- Test de l'ensemble des endpoints à l'aide de Postman.

Avantages et Limites du Monolithe

Avantages	Limites
Simplicité de développement et de déploiement	Scalabilité difficile à grande échelle
Débogage facilité (tout est dans un seul processus)	Couplage fort entre les modules
Bonne performance pour des charges modérées	Un bug peut impacter toute l'application
Idéal pour des équipes petites ou des prototypes	Migration vers les microservices complexe

Ce TP constitue un socle solide pour aborder par la suite les architectures distribuées (microservices), en comprenant d'abord les fondements d'une application monolithique bien structurée.